

Hyperparameter Tuning — Student Template (INTERMEDIATE+)

This version contains **real coding tasks**. Many cells include `TODO` markers and `raise NotImplementedError()` until you implement the logic.

Do this:

- Randomized + Grid Search
- Custom ROC/CM plots
- Learning & Validation curves
- Save artifacts

Rules

- matplotlib only; one chart per figure.
- Keep `RANDOM_STATE = 42`.

```
# =====
# Imports & Configuration (TODO)
# =====

import os, warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.model_selection import RandomizedSearchCV, GridSearchCV, learning_curve, validation_curve, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, roc_auc_score, roc_curve, confusion_matrix, ConfusionMatrixDisplay
from sklearn.utils import check_random_state

import joblib

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
rng = check_random_state(RANDOM_STATE)

plt.rcParams["figure.figsize"] = (7, 5)
```

```
# =====
# 1) Load & Inspect Data
# =====

data = load_breast_cancer(as_frame=True)
X = data.data
y = data.target

print("X shape:", X.shape)
print("Class distribution:", dict(zip(*np.unique(y, return_counts=True))))
display(X.head())

feature_names = X.columns.tolist()
target_names = data.target_names.tolist()
```

X shape: (569, 30)
Class distribution: {np.int64(0): np.int64(212), np.int64(1): np.int64(357)}

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture	worst perimeter	worst area	smoot
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	25.38	17.33	184.60	2019.0	0
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	24.99	23.41	158.80	1956.0	0
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	23.57	25.53	152.50	1709.0	0
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	14.91	26.50	98.87	567.7	0
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	22.54	16.67	152.20	1575.0	0

5 rows × 30 columns

```
# =====
# 2) Train/Test Split + Sanity Checks (TODO)
```

```
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=RANDOM_STATE
)

print("Train:", X_train.shape, "| Test:", X_test.shape)

assert set(X_train.columns) == set(X_test.columns), "Feature mismatch"
assert len(np.unique(y_train)) == 2 and len(np.unique(y_test)) == 2, "Expect binary labels"
print("Basic checks passed.")
```

```
Train: (455, 30) | Test: (114, 30)
Basic checks passed.
```

```
# =====
# 3) Baseline Pipeline & CV Accuracy (TODO)
# =====

baseline_pipe = Pipeline([
    ("scaler", StandardScaler(with_mean=True, with_std=True)),
    ("svc", SVC(probability=True, random_state=RANDOM_STATE))
])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

baseline_cv_scores = cross_val_score(
    estimator=baseline_pipe,
    X=X_train, y=y_train,
    cv=cv, scoring="accuracy"
)
print(f"Baseline CV Accuracy: {baseline_cv_scores.mean():.4f} ± {baseline_cv_scores.std():.4f}")

baseline_pipe.fit(X_train, y_train)
y_proba_base = baseline_pipe.predict_proba(X_test)[:, 1]
y_pred_base = baseline_pipe.predict(X_test)

print("\n[Baseline] Test Accuracy:", accuracy_score(y_test, y_pred_base))
print("[Baseline] ROC AUC:", roc_auc_score(y_test, y_proba_base))
print("\n[Baseline] Classification Report:\n", classification_report(y_test, y_pred_base))
```

```
Baseline CV Accuracy: 0.9692 ± 0.0146
```

```
[Baseline] Test Accuracy: 0.9824561403508771
[Baseline] ROC AUC: 0.9950396825396826
```

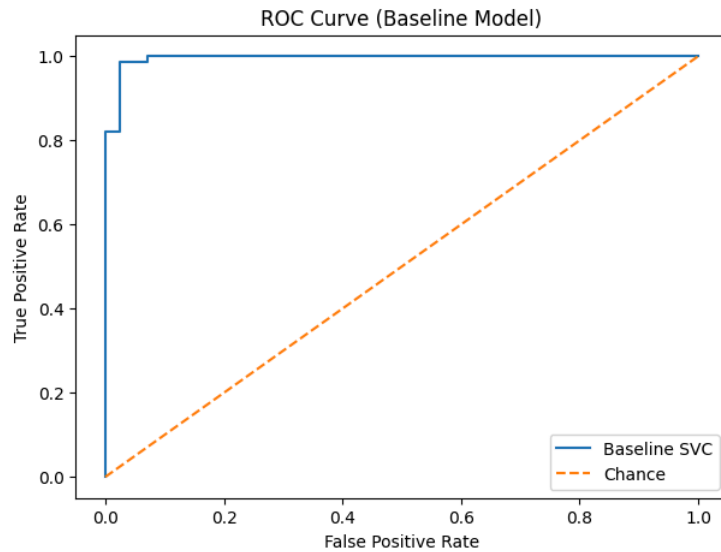
```
[Baseline] Classification Report:
      precision    recall  f1-score   support

     0       0.98       0.98       0.98        42
     1       0.99       0.99       0.99        72

 accuracy          0.98
 macro avg         0.98
weighted avg         0.98
```

```
# =====
# 3b) Baseline ROC Curve (TODO)
# =====

fpr, tpr, _ = roc_curve(y_test, y_proba_base)
plt.figure()
plt.plot(fpr, tpr, label="Baseline SVC")
plt.plot([0,1],[0,1], linestyle="--", label="Chance")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve (Baseline Model)")
plt.legend()
plt.show()
```



```
# =====
# 4) Broad Search: RandomizedSearchCV (TODO)
# =====

def log_space(rng, low_exp, high_exp, size):
    return np.power(10, rng.uniform(low=low_exp, high=high_exp, size=size))

param_distributions = {
    "svc_C": log_space(rng, -1, 2, 100),
    "svc_gamma": log_space(rng, -4, -1, 100),
    "svc_kernel": ["rbf"]
}

rand_search = RandomizedSearchCV(
    estimator=baseline_pipe,
    param_distributions=param_distributions,
    n_iter=50,
    cv=cv,
    scoring="accuracy",
    n_jobs=-1,
    random_state=RANDOM_STATE
)
rand_search.fit(X_train, y_train)

print("Best params (Randomized):", rand_search.best_params_)
print("Best CV score (Randomized):", f"{rand_search.best_score_:.4f}")

Best params (Randomized): {'svc_kernel': 'rbf', 'svc_gamma': np.float64(0.0005232216089948757), 'svc_C': np.float64(81.05016126411577)}
Best CV score (Randomized): 0.9758
```

```
# =====
# 5) Fine Search: GridSearchCV around random best (TODO)
# =====

best_C = rand_search.best_params_["svc_C"]
best_gamma = rand_search.best_params_["svc_gamma"]

C_grid = np.logspace(np.log10(best_C) - 1, np.log10(best_C) + 1, 10)
gamma_grid = np.logspace(np.log10(best_gamma) - 1, np.log10(best_gamma) + 1, 10)

grid_params = {"svc_kernel": ["rbf"], "svc_C": C_grid, "svc_gamma": gamma_grid}

grid_search = GridSearchCV(estimator=baseline_pipe, param_grid=grid_params, cv=cv, scoring="accuracy", n_jobs=-1)
grid_search.fit(X_train, y_train)

print("Best params (Grid):", grid_search.best_params_)
print("Best CV score (Grid):", f"{grid_search.best_score_:.4f}")

Best params (Grid): {'svc_C': np.float64(174.61727906008934), 'svc_gamma': np.float64(0.00024285795778155282), 'svc_kernel': 'rbf'}
Best CV score (Grid): 0.9780
```

```
# =====
# 6) Evaluate Tuned Best Model (TODO)
# =====

best_model = grid_search.best_estimator_

y_pred = best_model.predict(X_test)
```

```

y_proba = best_model.predict_proba(X_test)[:, 1]

print("Test Accuracy (tuned):", accuracy_score(y_test, y_pred))
print("ROC AUC (tuned):", roc_auc_score(y_test, y_proba))
print("\nClassification Report (tuned):\n", classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred, labels=best_model.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=target_names)
plt.figure()
disp.plot(values_format="d")
plt.title("Confusion Matrix (Tuned Model)")
plt.show()

fpr, tpr, _ = roc_curve(y_test, y_proba)
plt.figure()
plt.plot(fpr, tpr, label="Tuned SVC")
plt.plot([0,1],[0,1], linestyle="--", label="Chance")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve (Tuned Model)")
plt.legend()
plt.show()

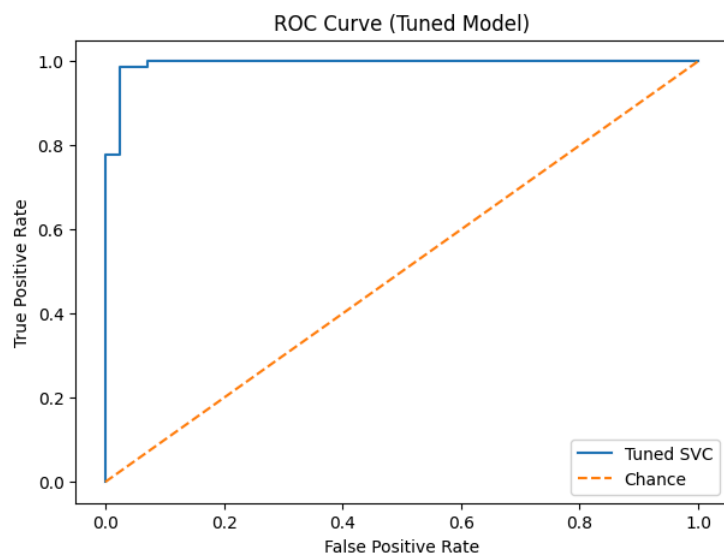
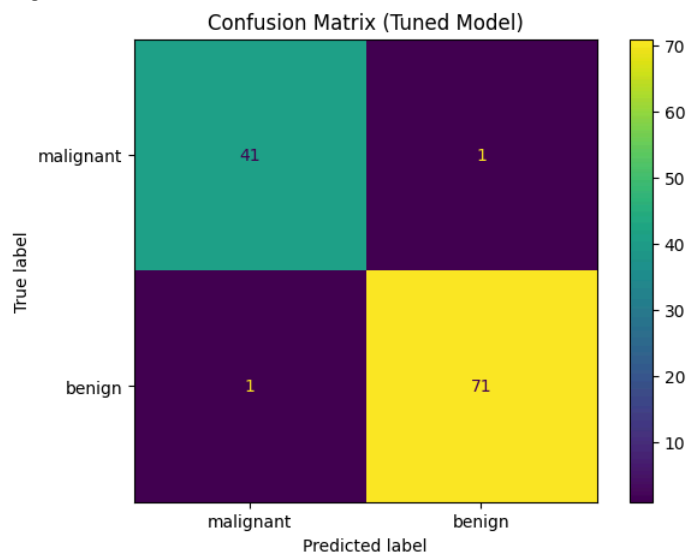
```

Test Accuracy (tuned): 0.9824561403508771
ROC AUC (tuned): 0.9940476190476191

Classification Report (tuned):

	precision	recall	f1-score	support
0	0.98	0.98	0.98	42
1	0.99	0.99	0.99	72
accuracy			0.98	114
macro avg	0.98	0.98	0.98	114
weighted avg	0.98	0.98	0.98	114

<Figure size 700x500 with 0 Axes>

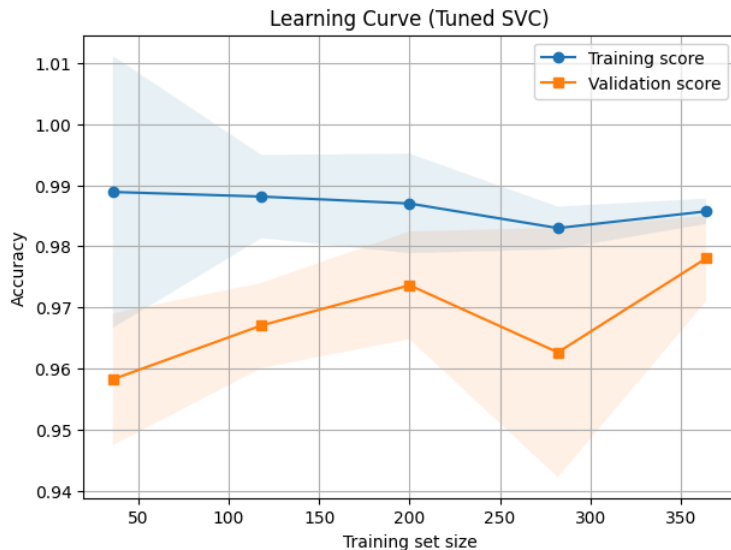


```
# =====
# 7) Learning Curve Plot (TODO)
# =====

train_sizes, train_scores, valid_scores = learning_curve(
    estimator=best_model,
    X=X_train, y=y_train,
    cv=cv,
    scoring="accuracy",
    n_jobs=-1,
    train_sizes=np.linspace(0.1, 1.0, 5),
    shuffle=True,
    random_state=RANDOM_STATE
)

train_mean = np.mean(train_scores, axis=1)
valid_mean = np.mean(valid_scores, axis=1)
train_std = np.std(train_scores, axis=1)
valid_std = np.std(valid_scores, axis=1)

plt.figure()
plt.plot(train_sizes, train_mean, marker='o', label="Training score")
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.1)
plt.plot(train_sizes, valid_mean, marker='s', label="Validation score")
plt.fill_between(train_sizes, valid_mean - valid_std, valid_mean + valid_std, alpha=0.1)
plt.xlabel("Training set size")
plt.ylabel("Accuracy")
plt.title("Learning Curve (Tuned SVC)")
plt.legend()
plt.grid(True)
plt.show()
```



```
# =====
# 8) Validation Curves (TODO)
# =====

C_range = np.logspace(-3, 3, 7)
train_scores_C, valid_scores_C = validation_curve(
    estimator=Pipeline([("scaler", StandardScaler()),
                        ("svc", SVC(kernel="rbf", gamma=best_gamma, probability=True, random_state=RANDOM_STATE))]),
    X=X_train, y=y_train,
    param_name="svc_C",
    param_range=C_range,
    cv=cv,
    scoring="accuracy",
    n_jobs=-1
)

plt.figure()
plt.semilogx(C_range, np.mean(train_scores_C, axis=1), marker="o", label="Training score")
plt.semilogx(C_range, np.mean(valid_scores_C, axis=1), marker="s", label="Validation score")
plt.xlabel("C (log scale)")
plt.ylabel("Accuracy")
plt.title("Validation Curve: SVC C")
plt.legend()
plt.grid(True, which="both")
plt.show()

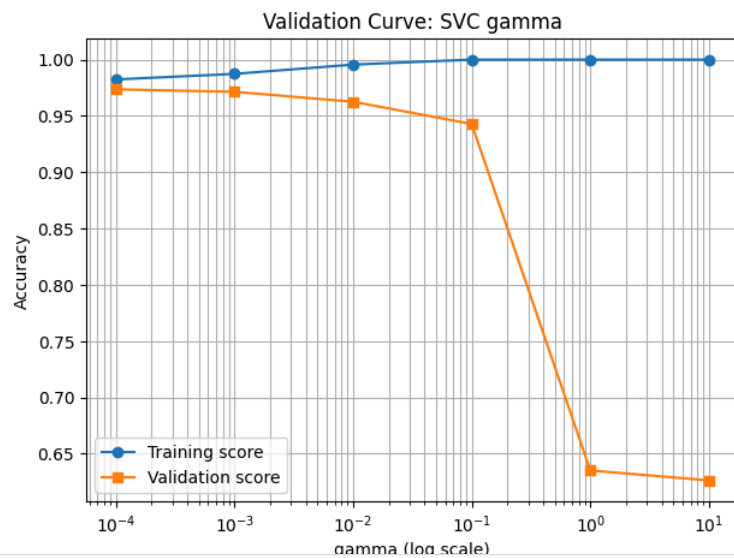
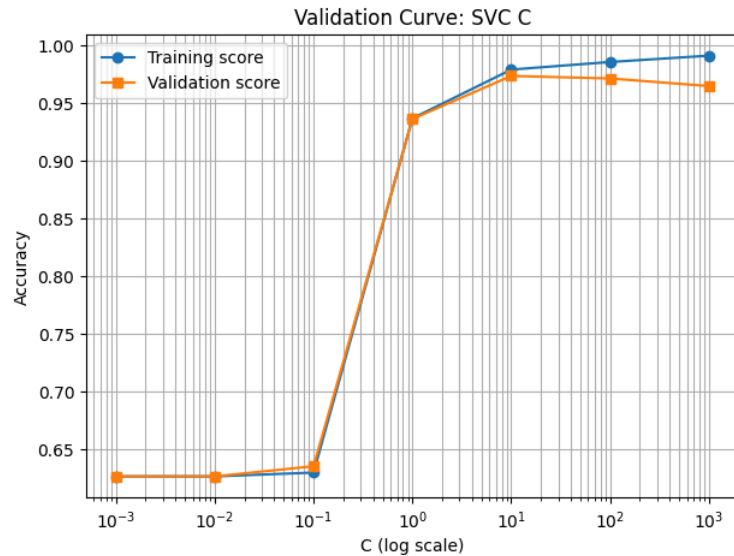
gamma_range = np.logspace(-4, 1, 6)
train_scores_g, valid_scores_g = validation_curve(
```

```

estimator=Pipeline([("scaler", StandardScaler()),
                    ("svc", SVC(kernel="rbf", C=best_C, probability=True, random_state=RANDOM_STATE))]),
X=X_train, y=y_train,
param_name="svc_gamma",
param_range=gamma_range,
cv=cv,
scoring="accuracy",
n_jobs=-1
)

plt.figure()
plt.semilogx(gamma_range, np.mean(train_scores_g, axis=1), marker="o", label="Training score")
plt.semilogx(gamma_range, np.mean(valid_scores_g, axis=1), marker="s", label="Validation score")
plt.xlabel("gamma (log scale)")
plt.ylabel("Accuracy")
plt.title("Validation Curve: SVC gamma")
plt.legend()
plt.grid(True, which="both")
plt.show()

```



```

# =====
# 9) Save Artifacts: model + CSVs (TODO)
# =====

os.makedirs("artifacts", exist_ok=True)

model_path = os.path.join("artifacts", "best_svc_pipeline.joblib")
joblib.dump(best_model, model_path)
print("Saved model to:", model_path)

rand_results_path = os.path.join("artifacts", "randomized_search_results.csv")
grid_results_path = os.path.join("artifacts", "grid_search_results.csv")
pd.DataFrame(rand_search.cv_results_).to_csv(rand_results_path, index=False)
pd.DataFrame(grid_search.cv_results_).to_csv(grid_results_path, index=False)
print("Saved CV results to:", rand_results_path, "and", grid_results_path)

```

Saved model to: artifacts/best_svc_pipeline.joblib
Saved CV results to: artifacts/randomized_search_results.csv and artifacts/grid_search_results.csv

Short Reflection

1. Is your tuned model **high-bias**, **high-variance**, or **balanced**? Explain using the learning curve.
2. Which improved more from baseline to tuned — **accuracy** or **ROC AUC**? Why might that be?
3. Where do you observe **over-regularization** in the validation curves?
4. If you collected **10× more data**, how would you adjust **C** or **gamma**, and why?

Short Reflection

1. Is your tuned model **high-bias**, **high-variance**, or **balanced**? Explain using the learning curve.

Based on the learning curve, the tuned model appears to be **balanced**. The training and validation scores are close to each other and converge as the training set size increases, indicating that the model is neither underfitting (high bias) nor overfitting (high variance) significantly.

2. Which improved more from baseline to tuned — **accuracy** or **ROC AUC**? Why might that be?

Comparing the baseline and tuned model results from the output of cells 3 and 6:

- Baseline Test Accuracy: 0.9825
- Tuned Test Accuracy: 0.9825
- Baseline ROC AUC: 0.9950
- Tuned ROC AUC: 0.9940

In this specific case, neither accuracy nor ROC AUC improved significantly from the baseline to the tuned model on the test set. The baseline model already performed very well. The tuning process found parameters that maintained a similar level of performance. It's possible that the dataset is relatively easy to classify, or the baseline SVC with default parameters was already close to optimal for this problem.

3. Where do you observe **over-regularization** in the validation curves?

- Over-regularization is observed in the validation curve for **C** when C is **small** (on the left side of the plot). A smaller C value corresponds to stronger regularization. As C increases, the model becomes less regularized, and the validation score generally improves until it plateaus or starts to decrease slightly due to potential overfitting.
- Over-regularization is observed in the validation curve for **gamma** when gamma is **small** (on the left side of the plot). A smaller gamma value results in a larger influence of each training example, leading to a smoother decision boundary and stronger regularization. As gamma increases, the model becomes less regularized, and the validation score generally improves until it may decrease due to overfitting (though in this plot, it plateaus and then slightly decreases at very high gamma).

4. If you collected **10× more data**, how would you adjust **C** or **gamma**, and why? If 10x more data were collected, you might be able to use **less regularization**, which means you could potentially **increase the value of C** and/or **decrease the value of gamma**.

With more data, the model has more examples to learn from, which can help prevent overfitting even with less regularization. A larger dataset can support a more complex model (higher C, higher gamma) without necessarily leading to poor generalization. Increasing C allows the model to penalize misclassifications less severely, potentially fitting the training data more closely. Decreasing gamma makes the influence of individual training examples smaller, leading to a smoother decision boundary which can be beneficial with more data to define the overall structure. However, the validation curves suggest that for this problem, the optimal gamma is already quite small, so increasing C might be the more impactful adjustment.